Use Case diagram - Design Decisions Document

1. View-First Navigation (Extension Pattern)

The system is designed so the user must **view** data before changing it. For example, instead of having direct buttons to edit or delete a task, the user must first open View Activity, which then optionally extends (<<extend>>) to Edit Activity or Delete Activity. This keeps the user interface clean and contextual.

2. Centralized Profile Hub

Instead of scattering settings everywhere, customization is grouped under a single path. A user goes to View Profile $\rightarrow$ Edit Profile $\rightarrow$ Customize Mochi or Customize app theme. This prevents accidental changes to settings and keeps all personalization in one secure place.

3. Calendar-Centric Scheduling

To manage tasks, the user must interact with View Calendar. The options to Create Activity or View Activity branch directly from the calendar view, forcing all tasks to be planned around a specific date and time rather than floating in a random list.

4. Enforced Routine Setup (<<include>>)

Unlike other optional features, View IdealDay explicitly **includes** (<<include>>) Set ideal day. This means looking at the daily routine view strictly requires the system to set up or validate those baseline hours (like sleep and work limits), ensuring the user's daily template is never empty, when he wants to see it.

5. Step-by-Step Goal Creation

The diagram enforces a strict order for goals: View Goals $\rightarrow$ Set global goals $\rightarrow$ Create subgoals. A user cannot just create a subtask out of nowhere; the system design forces them to build it under a main, global objective first.